👋

Getting started

📂 Спецификация Sprint №4: Социальное взаимодействие и бронирование подарков

1. Введение и цели спринта

1.1. Цель спринта

Расширение функциональности приложения механизмами социального взаимодействия. Реализация публичного доступа к спискам желаний по уникальной ссылке, внедрение системы бронирования подарков гостями без обязательной регистрации, а также внедрение механики «Сюрприз» — скрытия забронированных подарков от владельца списка для сохранения эффекта неожиданности.

2. Бэкенд: Модели данных и миграции

2.1. Расширение существующих моделей

2.1.1. Дополнение модели `Wishlist` (Список желаний)

В существующую модель `Wishlist` добавляется новое поле для генерации безопасных публичных ссылок.

Поле	Тип	Ограничения	Индекс	Описание
`public_id`	UUID	UNIQUE, NOT NULL, DEFAULT gen_random_uuid()	Да (btree)	Публичный идентификатор списка для шеринга

Требования:

- Для существующих записей поле должно быть автоматически заполнено через `gen_random_uuid()` при применении миграции.
- Поле не может быть `NULL`.
- Значение должно быть уникальным для каждой записи.

2.1.2. Новая модель `Reservation` (Бронирование)

Создаётся новая таблица для хранения информации о бронировании подарков гостями без регистрации.

Поле	Тип	Ограничения	Индекс	Описание
`id`	UUID	PK, DEFAULT gen_random_uuid()	Да (primary)	Уникальный идентификатор бронирования
`wishlist_item_id`	UUID	FK → wishlist_items(id), ON DELETE CASCADE, UNIQUE	Да (btree)	Ссылка на связь «список-товар»
`guest_name`	VARCHAR(100)	NOT NULL	Нет	Имя гостя, бронирующего подарок
`guest_email`	VARCHAR(255)	NOT NULL	Нет	Email гостя для связи
`reserved_at`	TIMESTAMP	NOT NULL, DEFAULT now()	Нет	Дата и время бронирования

Уникальные ограничения:

- `UNIQUE(wishlist_item_id)` — один товар в конкретном списке может быть забронирован только один раз.

Связи:

- Внешний ключ на таблицу `wishlist_items`.
- При удалении товара из списка бронирование удаляется каскадно.
- При удалении списка все его товары удаляются каскадно, что в свою очередь каскадно удаляет все связанные бронирования.

Дополнительные ограничения:

- Поле `guest_name` должно проходить валидацию: длина от 2 до 100 символов, только буквы и пробелы.
- Поле `guest_email` должно проходить валидацию формата email, длина ≤ 255 символов.

2.2. Миграции (Alembic)

Требования:

- Создать миграцию для добавления поля `public_id` в таблицу `wishlists`.
- Создать миграцию для создания таблицы `reservations`.
- Все миграции должны быть обратимыми (upgrade + downgrade).
- Миграции должны автоматически применяться при старте приложения в dev-режиме.

Порядок выполнения:

1. Добавление поля `public_id` в таблицу `wishlists` → миграция.
2. Создание таблицы `reservations` с внешним ключом и уникальным ограничением → миграция.

Проверка миграций:

- После применения миграций все существующие записи в `wishlists` должны иметь заполненное поле `public_id`.
- Таблица `reservations` должна быть пустой, но готовой к записи.

2.3. Требования к индексам (дополнение к существующим)

Таблица	Поле(я)	Тип индекса	Обоснование
`wishlists`	`public_id`	B-tree	Ускорение поиска списка по публичной ссылке при переходе гостя
`reservations`	`wishlist_item_id`	B-tree (UNIQUE)	Проверка статуса бронирования и защита от дублирования при параллельных запросах

Примечание: Индексы из предыдущих спринтов (`users.email`, `wishlists.user_id`, `wishlist_items.wishlist_id`, `wishlist_items.item_id`) сохраняются и используются в новом функционале.

2.4. Правила каскадного удаления

Действие	Результат
Удаление списка (`DELETE FROM wishlists`)	Каскадно удаляются все записи в `wishlist_items`, связанные с этим списком. Каскадно удаляются все записи в `reservations`, связанные с этими `wishlist_items`.
Удаление товара из списка (`DELETE FROM wishlist_items`)	Каскадно удаляется запись в `reservations`, связанная с этой связкой «список-товар».
Удаление пользователя (`DELETE FROM users`)	Каскадно удаляются все его списки, что запускает цепочку удалений выше.

2.5. Валидация данных на уровне приложения

Поле	Правило	Сообщение об ошибке
`public_id`	Автоматическая генерация, не требует ввода	—
`guest_name`	Обязательное, длина от 2 до 100 символов, только буквы и пробелы	"Имя должно содержать от 2 до 100 символов (только буквы и пробелы)"
`guest_email`	Обязательное, формат email, длина ≤ 255 символов	"Введите корректный email адрес"
`wishlist_item_id`	Обязательное, существующая связь «список-товар», не забронированная ранее	

3. Бэкенд: API эндпоинты для социального взаимодействия

3.1. Общие требования

- Эндпоинты делятся на две категории: **приватные** (требуют JWT-токен) и **публичные** (доступны без авторизации).
- Все ответы возвращают JSON.
- Валидация входных данных через Pydantic схемы.
- Для публичных эндпоинтов обязателен rate limiting.

3.2. Приватные эндпоинты (требуют JWT-токен)

3.2.1. GET /api/v1/wishlists/{wishlist_id}/share — Получить публичную ссылку на список

Назначение: генерация и получение публичной ссылки для шеринга списка.

Заголовки:

Ключ	Значение
`Authorization`	`Bearer <token>`

Параметры пути:

Параметр	Тип	Описание
`wishlist_id`	UUID	Идентификатор списка

Успешный ответ (200 OK):

Поле	Тип	Описание
`public_id`	string (UUID)	Публичный идентификатор списка
`share_url`	string	Полная публичная ссылка на список

```
{
  "public_id": "550e8400-e29b-41d4-a716-446655440000",
  "share_url": "http://localhost:3000/shared/550e8400-e29b-41d4-a716-446655440000"
}
```

Ошибки:

Код	Причина	Формат ответа
401	Отсутствует или невалиден токен	`{"detail": "Not authenticated"}`
403	Попытка получить ссылку на чужой список	`{"detail": "Access denied"}`
404	Список не найден	`{"detail": "Wishlist not found"}`

Логика:

- Проверить, что текущий пользователь является владельцем списка.
- Если у списка ещё нет `public_id`, сгенерировать его и сохранить в БД.

- Вернуть существующий или сгенерированный `public_id` и полную ссылку.

3.2.2. GET /api/v1/wishlists — Получить все списки пользователя (расширенный ответ)

Назначение: получение всех списков пользователя с дополнительной информацией для индикаторов прогресса.

Заголовки:

Ключ	Значение
`Authorization`	`Bearer <token>`

Успешный ответ (200 OK):

Поле	Тип	Описание
`id`	string (UUID)	Идентификатор списка
`title`	string	Название списка
`description`	string | null	Описание списка
`is_public`	boolean	Флаг публичности
`public_id`	string (UUID) | null	Публичный идентификатор (если есть)
`items_count`	integer	Общее количество товаров в списке
`reserved_count`	integer	Количество забронированных товаров
`purchased_count`	integer	Количество купленных товаров
`created_at`	timestamp	Дата создания
`share_url`	string | null	Публичная ссылка (если `public_id` существует)

```
[
  {
    "id": "123e4567-e89b-12d3-a456-426614174000",
    "title": "День рождения 2026",
    "description": "Что хочу получить на день рождения",
    "is_public": true,
    "public_id": "550e8400-e29b-41d4-a716-446655440000",
    "items_count": 10,
    "reserved_count": 3,
    "purchased_count": 2,
    "created_at": "2026-01-15T10:30:00Z",
    "share_url": "http://localhost:3000/shared/550e8400-e29b-41d4-a716-446655440000"
  }
]
```

Ошибки:

Код	Причина	Формат ответа
401	Отсутствует или невалиден токен	`{"detail": "Not authenticated"}`

3.2.3. GET /api/v1/wishlists/{wishlist_id} — Получить список с товарами (расширенный ответ)

Назначение: получение конкретного списка с учётом механики «Сюрприз».

Заголовки:

Ключ	Значение
`Authorization`	`Bearer <token>`

Параметры пути:

Параметр	Тип	Описание
`wishlist_id`	UUID	Идентификатор списка

Параметры запроса:

Параметр	Тип	Значение по умолчанию	Описание
`show_all`	boolean	`false`	Если `true`, показать все товары, включая забронированные

Успешный ответ (200 OK):

Поле	Тип	Описание
`id`	string (UUID)	Идентификатор списка
`title`	string	Название списка
`description`	string | null	Описание списка
`is_public`	boolean	Флаг публичности
`public_id`	string (UUID) | null	Публичный идентификатор
`share_url`	string | null	Публичная ссылка
`items`	array	Массив товаров (с учётом фильтрации)
`items[].id`	string (UUID)	Идентификатор товара
`items[].title`	string	Название товара
`items[].price`	number	Цена товара
`items[].currency`	string	Валюта
`items[].url`	string	Ссылка на товар
`items[].image_url`	string | null	Ссылка на изображение
`items[].priority`	integer	Приоритет (1-5)
`items[].note`	string | null	Личный комментарий
`items[].is_purchased`	boolean	Статус покупки
`items[].is_reserved`	boolean	Статус бронирования
`items[].reserved_by`	object | null	Информация о бронировании (если `show_all=true`)
`items[].reserved_by.guest_name`	string	Имя гостя
`items[].reserved_by.reserved_at`	timestamp	Дата бронирования
`items[].added_at`	timestamp	Дата добавления в список

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "title": "День рождения 2026",
  "description": "Что хочу получить на день рождения",
  "is_public": true,
  "public_id": "550e8400-e29b-41d4-a716-446655440000",
  "share_url": "http://localhost:3000/shared/550e8400-e29b-41d4-a716-446655440000",
  "items": [
    {
      "id": "111e2222-e33b-44d4-a555-666666777777",
      "title": "Наушники Sony WH-1000XM5",
      "price": 1299.00,
      "currency": "BYN",
      "url": "https://example.com/sony-headphones",
      "image_url": "https://example.com/image.jpg",
      "priority": 1,
      "note": "Чёрные",
```

```
      "is_purchased": false,
      "is_reserved": false,
      "reserved_by": null,
      "added_at": "2026-02-01T12:00:00Z"
    }
  ]
}
```

Ошибки:

Код	Причина	Формат ответа
401	Отсутствует или невалиден токен	`{"detail": "Not authenticated"}`
403	Попытка доступа к чужому приватному списку	`{"detail": "Access denied"}`
404	Список не найден	`{"detail": "Wishlist not found"}`

Логика фильтрации («Сюрприз»):

- Если пользователь является владельцем списка и `show_all=false`, то товары с `is_reserved=true` **исключаются** из ответа.
- Если пользователь является владельцем и `show_all=true`, то возвращаются все товары с информацией о бронировании.
- Если пользователь не является владельцем (чужой публичный список), то возвращаются все товары, но без приватных полей (`note`, `priority`).

3.3. Публичные эндпоинты (без авторизации)

3.3.1. GET /api/v1/public/wishlists/{public_id} — Получить публичный список

Назначение: просмотр содержимого списка по публичной ссылке без авторизации.

Параметры пути:

Параметр	Тип	Обязательность	Описание
`public_id`	UUID	Да	Публичный идентификатор списка

Успешный ответ (200 OK):

Поле	Тип	Описание
`id`	string (UUID)	Идентификатор списка
`title`	string	Название списка
`description`	string | null	Описание списка
`owner_name`	string	Имя владельца списка (email или часть до @)
`items`	array	Массив товаров
`items[].id`	string (UUID)	Идентификатор товара
`items[].title`	string	Название товара
`items[].price`	number	Цена товара
`items[].currency`	string	Валюта
`items[].url`	string	Ссылка на товар
`items[].image_url`	string | null	Ссылка на изображение
`items[].priority`	integer	Приоритет (1-5)
`items[].is_purchased`	boolean	Статус покупки
`items[].is_reserved`	boolean	Статус бронирования

Поле	Тип	Описание
`items[].reserved_by`	object | null	Информация о бронировании
`items[].reserved_by.guest_name`	string	Имя гостя, забронировавшего подарок

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "title": "День рождения 2026",
  "description": "Что хочу получить на день рождения",
  "owner_name": "ivan",
  "items": [
    {
      "id": "111e2222-e33b-44d4-a555-666666777777",
      "title": "Наушники Sony WH-1000XM5",
      "price": 1299.00,
      "currency": "BYN",
      "url": "https://example.com/sony-headphones",
      "image_url": "https://example.com/image.jpg",
      "priority": 1,
      "is_purchased": false,
      "is_reserved": true,
      "reserved_by": {
        "guest_name": "Анна"
      }
    }
  ]
}
```

Ошибки:

Код	Причина	Формат ответа
404	Список не найден или `public_id` не существует	`{"detail": "Wishlist not found"}`
403	Список существует, но `is_public = false`	`{"detail": "This wishlist is private"}`
429	Превышен лимит запросов	`{"detail": "Rate limit exceeded. Try again later."}`

Примечание:

- Поля `note` и `reserved_by.guest_email` **не возвращаются** в публичном ответе (приватные данные).
- `owner_name` формируется из email владельца (часть до символа `@`).

3.3.2. POST /api/v1/public/wishlists/{public_id}/items/{item_id}/reserve — Забронировать подарок

Назначение: бронирование подарка гостем без регистрации.

Параметры пути:

Параметр	Тип	Описание
`public_id`	UUID	Публичный идентификатор списка
`item_id`	UUID	Идентификатор товара

Заголовки:

Ключ	Значение
`Content-Type`	`application/json`

Тело запроса:

Поле	Тип	Валидация	Пример
`guest_name`	string	Длина от 2 до 100 символов, только буквы и пробелы	`"Анна Петрова"`
`guest_email`	string	Формат email, длина ≤ 255 символов	`"anna@example.com"`

Успешный ответ (201 Created):

Поле	Тип	Описание
`message`	string	Сообщение об успешном бронировании
`reservation_id`	string (UUID)	Идентификатор созданного бронирования
`item_title`	string	Название забронированного товара

```
{
  "message": "Подарок успешно забронирован",
  "reservation_id": "999e8888-e77b-66d6-b777-888888999999",
  "item_title": "Наушники Sony WH-1000XM5"
}
```

Ошибки:

Код	Причина	Формат ответа
404	Список или товар не найден	`{"detail": "Wishlist or item not found"}`
403	Список является приватным	`{"detail": "This wishlist is private"}`
409	Товар уже забронирован	`{"detail": "This item is already reserved"}`
400	Товар уже куплен	`{"detail": "Cannot reserve purchased item"}`
422	Невалидные данные (имя или email)	Pydantic validation error
429	Превышен лимит запросов	`{"detail": "Rate limit exceeded. Try again later."}`

Логика:

- Проверить, что список существует и `is_public = true`.
- Проверить, что товар существует и принадлежит указанному списку.
- Проверить, что товар не куплен (`is_purchased = false`).
- Проверить, что товар ещё не забронирован (отсутствует запись в `reservations`).
- Создать запись в таблице `reservations`.
- Вернуть успешный ответ.

3.4. Rate Limiting для публичных эндпоинтов

Эндпоинт	Лимит	Период	Ключ ограничения
`GET /public/wishlists/{public_id}`	30 запросов	1 минута	IP-адрес
`POST /public/wishlists/{public_id}/items/{item_id}/reserve`	5 запросов	1 минута	IP-адрес

При превышении лимита:

- HTTP статус: `429 Too Many Requests`
- Заголовок: `Retry-After: 60`
- Тело ответа: `{"detail": "Rate limit exceeded. Try again in 60 seconds."}`

3.5. Матрица обработки ошибок (общая)

Код	Сценарий	Тело ответа
400	Попытка забронировать уже купленный товар	`{"detail": "Cannot reserve purchased item"}`
401	Отсутствует или невалиден JWT токен (для приватных эндпоинтов)	`{"detail": "Not authenticated"}`
403	Попытка доступа к чужому приватному списку	`{"detail": "Access denied"}`
403	Попытка доступа к списку с `is_public = false` через публичный эндпоинт	`{"detail": "This wishlist is private"}`
404	Список, товар или `public_id` не найден	`{"detail": "Wishlist not found"}` / `{"detail": "Wishlist or item not found"}`
409	Попытка забронировать уже забронированный товар	`{"detail": "This item is already reserved"}`
422	Ошибка валидации входных данных	Pydantic validation error
429	Превышен rate limit	`{"detail": "Rate limit exceeded. Try again in 60 seconds."}`

4. Фронтенд: Макеты и верстка

4.1. Макеты Figma

Страница/компонент	Описание	Статус
Страницы		
Публичная страница списка (SharedView)	Страница просмотра списка по публичной ссылке без авторизации	К выполнению
Компоненты (модальные окна)		
Модальное окно бронирования	Окно с полями `guest_name` и `guest_email` для подтверждения бронирования	К выполнению

Примечание: Остальные страницы и компоненты (Dashboard, страница списка, форма создания/редактирования списка, форма добавления/редактирования подарка, диалог подтверждения удаления, карточки списка и товара, пустые состояния) были сверстаны в Sprint 2. В Sprint 4 они **дорабатываются** — добавляются кнопки «Поделиться», индикаторы прогресса и фильтры (задача SCRUM-37).

4.2. Публичная страница списка (SharedView)

URL: `/shared/{public_id}`

Назначение: просмотр содержимого списка желаний гостем без авторизации.

Функциональные требования:

Элемент	Требование
Заголовок	Название списка
Владелец	Отображение имени владельца (часть email до `@`)
Описание	Описание списка (если есть)
Список товаров	Отображение всех товаров в списке в режиме «только чтение»
Карточка товара (публичная)	Содержит: изображение (если есть), название, цену, валюту, приоритет (звёздочки 1-5), статус (свободен/забронирован/куплен)
Кнопка «Забронировать»	Отображается только для свободных товаров. Открывает модальное окно бронирования
Плашка «Занято»	Отображается для забронированных товаров. Содержит имя гостя, забронировавшего подарок
Плашка «Куплено»	Отображается для купленных товаров
Адаптивность	На мобильных устройствах карточки располагаются вертикально, на десктопе — сеткой (2-3 колонки)

4.3. Модальное окно бронирования

Тип: Модальное окно

Условие открытия: нажатие на кнопку «Забронировать» в карточке свободного товара на публичной странице.

Структура модального окна:

Элемент	Описание
Заголовок	«Забронировать подарок»
Название товара	Отображается под заголовком для подтверждения
Поле «Ваше имя»	Обязательное поле для ввода имени гостя
Поле «Ваш email»	Обязательное поле для ввода email гостя
Кнопка «Подтвердить бронирование»	Акцентная, отправляет запрос
Кнопка «Отмена»	Нейтральная, закрывает окно
Крестик закрытия	В правом верхнем углу

4.4. Поля формы бронирования

Поле	Тип	Валидация	Атрибуты
Ваше имя	text	Длина от 2 до 100 символов, только буквы и пробелы	`placeholder="Иван Петров"`, `autocomplete="name"`
Ваш email	email	Формат email, длина ≤ 255 символов	`placeholder="ivan@example.com"`, `autocomplete="email"`

4.5. Клиентская валидация формы бронирования

Поле	Правило	Сообщение об ошибке
Ваше имя	Не пустое	«Укажите ваше имя»
Ваше имя	Длина от 2 до 100 символов	«Имя должно содержать от 2 до 100 символов»
Ваше имя	Только буквы и пробелы	«Имя может содержать только буквы и пробелы»
Ваш email	Не пустой	«Укажите email»
Ваш email	Соответствует формату `user@domain.com`	«Введите корректный email адрес»

Валидация должна выполняться:

- В реальном времени (onChange или onBlur) — после первого касания поля.
- При отправке формы (финальная проверка).

4.6. Поведение формы бронирования

Состояние	Поведение
Default	Кнопка «Подтвердить бронирование» активна только при валидности всех полей
Загрузка (submitting)	Кнопка меняет текст на «Бронирование...», становится неактивной, поля блокируются
Успех	Модальное окно закрывается. Карточка товара обновляется: кнопка «Забронировать» заменяется на плашку «Занято» с именем гостя. Показывается toast-уведомление: «Подарок успешно забронирован!»
Ошибка 409 (уже забронирован)	Модальное окно закрывается. Карточка товара обновляется до состояния «Занято». Показывается toast-уведомление: «К сожалению, этот подарок уже забронировал другой гость»
Ошибка 400 (уже куплен)	Модальное окно закрывается. Карточка товара обновляется до состояния «Куплено». Показывается toast-уведомление: «Этот подарок уже куплен»
Ошибка 429 (rate limit)	Показывается сообщение под формой: «Слишком много попыток. Попробуйте позже»
Ошибка сети	Показывается сообщение под формой: «Не удалось отправить запрос. Проверьте подключение к интернету»
Отмена	При нажатии «Отмена», крестик, клик на оверлей или Escape — окно закрывается без отправки, поля очищаются

4.7. Карточка товара на публичной странице (компонент SharedGiftCard)

Где используется: Публичная страница списка (SharedView)

Функциональные требования:

Элемент	Требование
Изображение	Если есть — отображается превью. Если нет — плейсхолдер
Название	Жирный шрифт, 2 строки с обрезкой
Цена и валюта	Формат «1 299 BYN» или «$ 49.99»
Приоритет	Звёздочки (1-5)
Статус «Свободен»	Отображается кнопка «Забронировать»
Статус «Занят»	Отображается плашка «Занято: {имя гостя}». Кнопка бронирования скрыта
Статус «Куплен»	Отображается плашка «Куплено». Кнопка бронирования скрыта, карточка может быть визуально приглушена (серый оттенок)
Hover-эффект	Тень или подсветка при наведении (только для свободных товаров)

4.8. Доработка существующих страниц (в рамках задачи SCRUM-37)

В Sprint 4 существующие страницы, сверстанные в Sprint 2, получают следующие доработки:

Страница	Доработка
Dashboard	Добавляется индикатор прогресса для каждого списка (например, прогресс-бар «5/10 подарков забронировано»). Добавляется кнопка «Поделиться» на карточке списка (если `is_public = true`)
Страница списка (ListView)	Добавляется кнопка «Поделиться» в заголовке. Добавляются фильтры: «Все», «Свободные», «Забронированные», «Купленные». Добавляется чекбокс «Показать забронированные» (режим «Сюрприз»)

4.9. Адаптивность

Устройство	Требование
Мобильные (375px)	Карточки товаров располагаются вертикально в одну колонку. Модальное окно бронирования занимает 90% ширины экрана
Планшеты (768px)	Карточки товаров располагаются в 2 колонки. Модальное окно шириной 500-600px
Десктоп (1024px+)	Карточки товаров располагаются в 3 колонки. Модальное окно шириной 500px

5. Фронтенд: Интеграция с API

5.1. API клиент — расширение

В Sprint 4 существующий API клиент дополняется новыми функциями для работы с социальным функционалом.

Функция	Эндпоинт	Описание
`getShareLink(wishlistId)`	GET /api/v1/wishlists/{wishlist_id}/share	Получить публичную ссылку на список
`getPublicWishlist(publicId)`	GET /api/v1/public/wishlists/{public_id}	Получить публичный список по ссылке (без авторизации)
`reserveItem(publicId, itemId, data)`	POST /api/v1/public/wishlists/{public_id}/items/{item_id}/reserve	Забронировать подарок (без авторизации)

Примечание: Существующие функции `getWishlists()` и `getWishlistById()` дорабатываются на бэкенде для возврата дополнительных полей (`public_id`, `share_url`, `reserved_count`, `is_reserved`), но сигнатуры вызовов остаются без изменений.

5.2. Управление состоянием

В Sprint 4 существующие хуки дополняются новыми возможностями.

Хук/контекст	Назначение	Изменения в Sprint 4
`useWishlists`	Управление списками пользователя	Добавляется метод `getShareLink` для получения публичной ссылки
`useWishlist`	Управление конкретным списком и его товарами	Добавляется поддержка параметра `showAll` для режима «Сюрприз»
`usePublicWishlist`	**Новый хук**	Управление публичным списком (загрузка данных по `public_id`)
`useReservation`	**Новый хук**	Управление бронированием (отправка формы, обработка состояний)

Требования:

- После успешного бронирования — автоматически обновлять данные на публичной странице без перезагрузки.
- Показывать индикатор загрузки во время выполнения операций.
- Обрабатывать ошибки и показывать понятные сообщения.

5.3. Интеграция публичной страницы (SharedView)

Алгоритм загрузки страницы:

1. Извлечь `public_id` из параметров URL (`/shared/{public_id}`).
2. Вызвать `getPublicWishlist(public_id)`.
3. На время загрузки отображать скелетон страницы (заглушки карточек).
4. При успешном ответе — отобразить список и товары.
5. При ошибке:
   - **404** — отобразить сообщение «Список не найден».
   - **403** — отобразить сообщение «Этот список является приватным».
   - **429** — отобразить сообщение «Слишком много запросов. Попробуйте позже».

5.4. Интеграция бронирования

Алгоритм бронирования:

1. Пользователь нажимает кнопку «Забронировать» на карточке свободного товара.
2. Открывается модальное окно с формой (имя, email).
3. Пользователь заполняет поля и нажимает «Подтвердить бронирование».
4. Вызывается `reserveItem(publicId, itemId, { guest_name, guest_email })`.
5. На время запроса:
   - Кнопка блокируется с текстом «Бронирование...».
   - Поля формы блокируются.
6. При успешном ответе (201):
   - Модальное окно закрывается.
   - Карточка товара обновляется до состояния «Занято» с именем гостя.
   - Показывается toast-уведомление «Подарок успешно забронирован!».
7. При ошибке:

- **409 (уже забронирован)** — модальное окно закрывается, карточка обновляется до «Занято», toast «К сожалению, этот подарок уже забронировал другой гость».
- **400 (уже куплен)** — модальное окно закрывается, карточка обновляется до «Куплено», toast «Этот подарок уже куплен».
- **422 (валидация)** — ошибки отображаются под соответствующими полями формы.
- **429 (rate limit)** — сообщение под формой «Слишком много попыток. Попробуйте позже».

5.5. Интеграция кнопки «Поделиться» (Dashboard и ListView)

Алгоритм получения и копирования ссылки:

1. Пользователь нажимает кнопку «Поделиться» на карточке списка (Dashboard) или в заголовке списка (ListView).
2. Вызывается `getShareLink(wishlistId)`.
3. На время запроса кнопка переходит в состояние загрузки (спиннер).
4. При успешном ответе:
   - Ссылка `share_url` копируется в буфер обмена.
   - Показывается toast-уведомление «Ссылка скопирована в буфер обмена».
5. При ошибке:
   - Показывается toast-уведомление «Не удалось получить ссылку. Попробуйте позже».

Примечание: Если список имеет `is_public = false`, перед запросом к API необходимо либо:

- Автоматически обновить флаг `is_public` на `true` через `updateWishlist`.
- Показать диалог с предложением сделать список публичным.

5.6. Интеграция режима «Сюрприз» (ListView)

Алгоритм переключения режима:

1. На странице списка (ListView) добавляется чекбокс или тумблер «Показать забронированные».
2. По умолчанию тумблер выключен (`show_all = false`) — забронированные подарки скрыты.
3. При включении тумблера (`show_all = true`) выполняется повторный запрос `getWishlistById(id, { show_all: true })`.
4. Список товаров обновляется — забронированные подарки становятся видны с плашкой «Занято: {имя гостя}».

5.7. Обработка ошибок (общая)

Сценарий	Сообщение пользователю
Не удалось загрузить публичный список	«Не удалось загрузить список. Проверьте ссылку или попробуйте позже.»
Список не найден (404)	«Список не найден. Возможно, ссылка устарела.»
Список приватный (403)	«Этот список является приватным. Владелец не разрешил публичный доступ.»
Не удалось забронировать подарок	«Не удалось забронировать подарок. Проверьте подключение к интернету.»
Подарок уже забронирован (409)	«К сожалению, этот подарок уже забронировал другой гость.»
Подарок уже куплен (400)	«Этот подарок уже куплен.»
Слишком много запросов (429)	«Слишком много попыток. Попробуйте позже.»
Не удалось скопировать ссылку	«Не удалось скопировать ссылку. Скопируйте её вручную.»

5.8. Отмена устаревших запросов (Request Cancellation)

- При переходе между страницами или повторных запросах предыдущие незавершённые запросы должны отменяться через `AbortController`.

- При закрытии модального окна бронирования до завершения запроса — запрос отменяется.

5.9. Обновление кэша и состояния

- После успешного бронирования публичная страница **не перезагружается полностью** — обновляется только состояние конкретной карточки товара.
- После изменения флага `is_public` через форму редактирования списка — кнопка «Поделиться» должна появиться/исчезнуть без перезагрузки страницы.
- После копирования ссылки состояние кнопки возвращается в исходное.

6. Тестирование

6.1. Unit-тесты бэкенда (pytest)

6.1.1. Общие требования

- Тесты должны быть асинхронными (`@pytest.mark.asyncio`).
- Используется `httpx.AsyncClient` для эмуляции HTTP-запросов.
- Для приватных эндпоинтов аутентификация передается через фикстуру `auth_headers`.
- Для публичных эндпоинтов заголовок `Authorization` не передаётся.
- Каждый тест-кейс проверяет только один сценарий.

6.1.2. Публичный доступ к списку

ID	Название теста	Предусловие	Шаги	Ожидае
P-01	Получение публичного списка с `is_public = true`	Список создан, `is_public = true`	`GET /api/v1/public/wishlists/{public_id}`	`200 OK`, содержи массив
P-02	Получение публичного списка с `is_public = false`	Список создан, `is_public = false`	`GET /api/v1/public/wishlists/{public_id}`	`403 Forb is priva`
P-03	Получение публичного списка с несуществующим `public_id`	—	`GET /api/v1/public/wishlists/{невалидный-uuid}`	`404 Not found"}`
P-04	Проверка отсутствия приватных полей в публичном ответе	Список с товарами, у товара есть `note`	`GET /api/v1/public/wishlists/{public_id}`	В ответе поля `no`
P-05	Проверка формирования `owner_name`	Список создан пользователем `ivan@example.com`	`GET /api/v1/public/wishlists/{public_id}`	`owner_na`

6.1.3. Бронирование подарка

ID	Название теста	Предусловие	Шаги
R-01	Бронирование свободного подарка	Публичный список со свободным товаром	`POST /api/v1/public/wishlists/{public_id}/items/{item_id}/reserve` валидными `guest_name`, `guest_email`
R-02	Бронирование уже забронированного подарка	Товар уже забронирован другим гостем	`POST .../reserve` с валидными данными
R-03	Бронирование купленного подарка	Товар имеет `is_purchased = true`	`POST .../reserve` с валидными данными

ID	Название теста	Предусловие	Шаги
R-04	Бронирование с невалидным `guest_name` (пустое)	Публичный список со свободным товаром	`POST .../reserve` с `{"guest_name": "", "guest_email": "test@example.com"}`
R-05	Бронирование с невалидным `guest_name` (короткое)	Публичный список со свободным товаром	`POST .../reserve` с `{"guest_name": "A", ...}`
R-06	Бронирование с невалидным `guest_name` (цифры/спецсимволы)	Публичный список со свободным товаром	`POST .../reserve` с `{"guest_name": "Anna123", ...}`
R-07	Бронирование с невалидным `guest_email`	Публичный список со свободным товаром	`POST .../reserve` с `{"guest_email": "not-an-email"}`
R-08	Бронирование с несуществующим `public_id`	—	`POST /api/v1/public/wishlists/{невалидный-uuid}/items/{item_id}/reserve`
R-09	Бронирование с несуществующим `item_id`	Публичный список	`POST .../items/{невалидный-uuid}/reserve`
R-10	Бронирование товара из чужого списка (item_id не принадлежит списку)	Два публичных списка	`POST .../reserve` с `item_id` из другого списка

6.1.4. Приватные эндпоинты (шеринг и «Сюрприз»)

ID	Название теста	Предусловие	Шаги	Ожидаемый результат
S-01	Получение ссылки на свой список	Пользователь А авторизован, создал список	`GET /api/v1/wishlists/{wishlist_id}/share`	`200 OK`, содержит `public_id` и `share_url`
S-02	Получение ссылки на чужой список	Пользователь Б создал список	Пользователь А вызывает `GET .../share` для списка Б	`403 Forbidden`
S-03	Просмотр своего списка без забронированных (`show_all=false`)	Владелец списка, есть забронированный товар	`GET /api/v1/wishlists/{id}`	Забронированный товар **отсутствует** в `items`
S-04	Просмотр своего списка с забронированными (`show_all=true`)	Владелец списка, есть забронированный товар	`GET /api/v1/wishlists/{id}?show_all=true`	Забронированный товар **присутствует**, содержит `is_reserved: true` и `reserved_by`
S-05	Просмотр чужого публичного списка (владельцем не является)	Пользователь Б создал публичный список	Пользователь А вызывает `GET /api/v1/wishlists/{id}`	`200 OK`, но `note` и `guest_email` скрыты
S-06	Просмотр чужого приватного списка	Пользователь Б создал приватный список	Пользователь А вызывает `GET /api/v1/wishlists/{id}`	`403 Forbidden`

6.1.5. Rate Limiting

ID	Название теста	Предусловие	Шаги	Ожидаемый результат
RL-01	Превышение лимита на просмотр публичного	—	31 запрос `GET /public/wishlists/{public_id}` за 1 минуту	31-й запрос возвращает `429 Too`

ID	Название теста	Предусловие	Шаги	Ожидаемый результат
	списка		с одного IP	`Many Requests`
RL-02	Превышение лимита на бронирование	Публичный список	6 запросов `POST .../reserve` за 1 минуту с одного IP	6-й запрос возвращает `429 Too Many Requests`

6.1.6. Каскадное удаление

ID	Название теста	Предусловие	Шаги	Ожидаемый результат
C-01	Удаление брони при удалении товара из списка	Товар забронирован	`DELETE /api/v1/wishlists/{wl_id}/items/{item_id}`	Бронь удалена из таблицы `reservations`
C-02	Удаление брони при удалении списка	Товар забронирован	`DELETE /api/v1/wishlists/{wl_id}`	Бронь удалена каскадно

6.2. Postman коллекция (дополнение)

В существующую коллекцию добавляются новые запросы:

Эндпоинт	Тесты
`GET /wishlists/{id}/share`	Статус 200, наличие `public_id` и `share_url`. Проверка 403 для чужого списка
`GET /wishlists` (расширенный)	Статус 200, наличие полей `public_id`, `share_url`, `reserved_count`, `purchased_count`
`GET /wishlists/{id}` (с `show_all`)	Статус 200, проверка фильтрации забронированных товаров при `show_all=false` и `show_all=true`
`GET /public/wishlists/{public_id}`	Статус 200 для публичного списка, 403 для приватного, 404 для несуществующего
`POST /public/wishlists/{public_id}/items/{item_id}/reserve`	Статус 201 для успешного бронирования, 409 для повторного, 400 для купленного, 422 для невалидных данных

6.3. Сценарии для Collection Runner

1. **Полный цикл публичного доступа:**
   - Создать список с `is_public = true`.
   - Получить `share_url` через `GET /wishlists/{id}/share`.
   - Перейти по публичной ссылке → `GET /public/wishlists/{public_id}`.
   - Забронировать товар → `POST .../reserve`.
   - Проверить, что повторное бронирование возвращает 409.
2. **Проверка режима «Сюрприз»:**
   - Создать список, добавить товар.
   - Забронировать товар через публичный эндпоинт.
   - Проверить, что владелец не видит товар при `show_all=false`.
   - Проверить, что владелец видит товар при `show_all=true`.
3. **Проверка прав доступа:**
   - Пользователь А создаёт приватный список.
   - Пользователь Б пытается получить список через приватный эндпоинт → 403.
   - Пользователь Б пытается получить список через публичный эндпоинт → 403.

6.4. Сценарии ручного тестирования (QA)

6.4.1. Сквозные сценарии (E2E)

ID	Сценарий	Шаги	Ожидаемый результат
E2E-01	**Гость просматривает публичный список**	1. Владелец копирует публичную ссылку2. Гость переходит по ссылке	Отображается список с товарами, кнопки «Забронировать» для свободных товаров
E2E-02	**Гость бронирует подарок**	1. Гость нажимает «Забронировать»2. Заполняет имя и email3. Подтверждает	Карточка обновляется до «Занято: {имя}», toast «Подарок успешно забронирован!»
E2E-03	**Гость пытается забронировать уже занятый подарок**	1. Другой гость уже забронировал товар2. Гость нажимает «Забронировать»3. Заполняет форму4. Подтверждает	Модальное окно закрывается, карточка обновляется до «Занято», toast о том, что подарок уже занят
E2E-04	**Владелец не видит забронированные подарки**	1. Гость бронирует подарок2. Владелец заходит в свой список	Забронированный подарок не отображается
E2E-05	**Владелец включает режим «Показать всё»**	1. Владелец заходит в список2. Включает тумблер «Показать забронированные»	Забронированный подарок отображается с плашкой «Занято: {имя}»
E2E-06	**Владелец копирует ссылку на список**	1. Владелец нажимает «Поделиться» на карточке списка	Ссылка копируется в буфер, toast «Ссылка скопирована»
E2E-07	**Гость переходит по приватной ссылке**	1. Владелец делает список приватным2. Гость переходит по старой ссылке	Отображается сообщение «Этот список является приватным»

6.4.2. Кроссбраузерное тестирование

Браузер	Устройство	Проверяемые сценарии
Chrome (последняя версия)	Desktop	Все E2E сценарии
Safari (последняя версия)	Desktop	Все E2E сценарии
Firefox (последняя версия)	Desktop	Все E2E сценарии
Chrome (мобильный)	Android	Просмотр публичного списка, бронирование
Safari (мобильный)	iOS	Просмотр публичного списка, бронирование

6.4.3. Тестирование адаптивности

Разрешение	Проверяемые элементы
375px (мобильные)	Карточки товаров в 1 колонку, модальное окно бронирования 90% ширины
768px (планшеты)	Карточки товаров в 2 колонки
1024px+ (десктоп)	Карточки товаров в 3 колонки

6.5. Тестирование безопасности

Проверка	Действие	Ожидание
IDOR-01	Гость пытается забронировать товар из чужого списка, подменив `public_id`	`404 Not Found`
IDOR-02	Гость пытается получить приватные поля (`note`, `guest_email`) через публичный эндпоинт	Поля отсутствуют в ответе
IDOR-03	Пользователь А пытается получить ссылку на список пользователя Б	`403 Forbidden`
Rate Limit-01	31 запрос к публичному эндпоинту за 1 минуту	`429 Too Many Requests`
Rate Limit-02	6 запросов на бронирование за 1 минуту	`429 Too Many Requests`

Проверка	Действие	Ожидание
XSS-01	Ввод HTML/JS в поле `guest_name` при бронировании	Данные экранируются, XSS не выполняется